

2.3.4 FORMAL DEFINITIONS OF MESSAGES

2.3.4.1 Encoding/Decoding Rules

2.3.4.1.1 A CPDLC-air-ASE shall be capable of encoding AircraftPDUs APDUs and decoding GroundPDUs APDUs as defined in the ASN.1 module CPDLCAPDUsVersion1 specified in 2.3.4.2.

Note.— The ProtectedUplinkMessage and ProtectedDownlinkMessage ASN.1 types both contain the CPDLCMessage type, which resolves to a BIT STRING type. Even though this BIT STRING is specified as containing a PER Encoded ATCUplinkMessage or a PER Encoded ATCDownlinkMessage, there is no requirement on the CPDLC-air-ASE to verify this. The CPDLC-air-user is responsible for encoding a valid ATCDownlinkMessage and for verifying the correctness of a received ATCUplinkMessage.

2.3.4.1.2 The CPDLC-air-user shall be capable of encoding ATCDownlinkMessage and decoding ATCUplinkMessage types, as defined in the ASN.1 module CPDLCMessageSetVersion1 specified in 2.3.4.3.

2.3.4.1.3 A CPDLC-ground-ASE shall be capable of encoding GroundPDUs APDUs and decoding AircraftPDUs APDUs as defined in the ASN.1 module CPDLCAPDUsVersion1 specified in 2.3.4.2.

Note 1.— The ProtectedUplinkMessage and ProtectedDownlinkMessage ASN.1 types both contain the CPDLCMessage type, which resolves to a BIT STRING type. Even though this BIT STRING is specified as containing a PER Encoded ATCUplinkMessage or a PER Encoded ATCDownlinkMessage, there is no requirement on the CPDLC-ground-ASE to verify this. The CPDLC-ground-user is responsible for encoding a valid ATCUplinkMessage and for verifying the correctness of a received ATCDownlinkMessage.

Note 2.— The ForwardMessage ASN.1 type contains the uplink or downlink CPDLC Message Element data type, which both resolve to a BIT STRING type. Even though this BIT STRING is specified as containing a PER Encoded ATCUplinkMessageData or a PER Encoded ATCDownlinkMessageData, there is no requirement on the CPDLC-ground-ASE to verify this. The CPDLC-ground-user is responsible for encoding and verifying the correctness of valid ATCDownlinkMessageData and ATCUplinkMessageData.

2.3.4.1.4 The CPDLC-ground-user shall be capable of encoding ATCUplinkMessage and decoding ATCDownlinkMessage types, as defined in the ASN.1 module CPDLCMessageSetVersion1 specified in 2.3.4.3.

2.3.4.1.5 The CPDLC-ground-user shall be capable of encoding and decoding ATCUplinkMessageData and ATCDownlinkMessageData types, as defined in the ASN.1 module CPDLCMessageSetVersion1 specified in 2.3.4.3.

2.3.4.2 CPDLC Message ASN.1 Abstract Syntax

2.3.4.2.1 The abstract syntax of the CPDLC protocol data units shall comply with the description contained in the ASN.1 module CPDLCAPDUsVersion1 conforming to ISO/IEC 8824, as defined in this section.

CPDLCAPDUsVersion1 DEFINITIONS::=

BEGIN

IMPORTS

 DateTimeGroup,
 AircraftFlightIdentification,
 AircraftAddress
 FROM CPDLCMessageSetVersion1;

-- Ground Generated Messages - Top level

GroundPDUs ::= CHOICE

{
 abortUser [0] CPDLCUserAbortReason,
 abortProvider [1] CPDLCProviderAbortReason,
 startup [2] ProtectedUplinkMessage,
 send [3] ProtectedUplinkMessage,
 forward [4] ATCForwardMessage,
 forwardresponse [5] ATCForwardResponse,
 ...
}

ProtectedUplinkMessage ::= SEQUENCE

{
 algorithmIdentifier [0] AlgorithmIdentifier OPTIONAL,
 protectedMessage [1] CPDLCMessage OPTIONAL,
 -- PER encoded ATCUplinkMessage
 -- (see Module CPDLCMessageSetVersion1)
 integrityCheck [2] BIT STRING,
 ...
}

ATCForwardMessage ::= SEQUENCE

{
 forwardHeader ForwardHeader,
 forwardMessage ForwardMessage
}

ForwardHeader ::= SEQUENCE

```
{
  dateTime          DateTimeGroup,
  aircraftID        AircraftFlightIdentification,
  aircraftAddress   AircraftAddress
}
```

ForwardMessage ::= CHOICE

```
{
  upElementIDs      [0]    BIT STRING,
                                --PER encoded ATCUplinkMessageData,
                                -- (see Module CPDLCMessageSetVersion1)
  downElementIDs    [1]    BIT STRING
                                --PER encoded ATCDownlinkMessageData,
                                -- (see Module CPDLCMessageSetVersion1) }
}
```

ATCForwardResponse ::= ENUMERATED

```
{
  success              (0),
  service-not-supported (1),
  version-not-equal    (2),
  ...
}
```

 -- Aircraft Generated Messages - Top level

AircraftPDUs ::= CHOICE

```
{
  abortUser      [0]    CPDLCUserAbortReason,
  abortProvider  [1]    CPDLCProviderAbortReason,
  startdown      [2]    StartDownMessage,
  send           [3]    ProtectedDownlinkMessage,
  ...
}
```

StartDownMessage ::= SEQUENCE

```
{
  mode              Mode DEFAULT cpdlc,
  startDownlinkMessage ProtectedDownlinkMessage
}
```

Mode ::= ENUMERATED

```
{
  cpdlc          (0),
  dsc            (1)
}
```

ProtectedDownlinkMessage ::= SEQUENCE

```
{
  algorithmIdentifier [0] AlgorithmIdentifier OPTIONAL,
  protectedMessage    [1] CPDLCMessage OPTIONAL,
                      --PER encoded ATCDownlinkMessage,
                      -- (see Module CPDLCMessageSetVersion1)
  integrityCheck      [2] BIT STRING,
  ...
}
```

 -- Uplink and Downlink messages - Common Elements

AlgorithmIdentifier ::= RELATIVE-OID --root is {icao-arc atn-algorithms(9)}

CPDLCMessage ::= BIT STRING

CPDLCUserAbortReason ::= ENUMERATED

```
{
  undefined                                (0),
  no-message-identification-numbers-available (1),
  duplicate-message-identification-numbers   (2),
  no-longer-next-data-authority              (3),
  current-data-authority-abort                (4),
  commanded-termination                     (5),
  invalid-response                           (6),
  time-out-of-synchronisation                 (7),
  unknown-integrity-check                    (8),
  validation-failure                         (9),
  unable-to-decode-message                   (10),
  invalid-pdu                                (11),
  invalid-CPDLC-message                      (12),
  ...
}
```

CPDLCProviderAbortReason ::= ENUMERATED

```
{
  timer-expired                (0),
  undefined-error               (1),
  invalid-PDU                   (2),
  protocol-error                (3),
}
```

communication-service-error	(4),
communication-service-failure	(5),
invalid-QOS-parameter	(6),
expected-PDU-missing	(7),
...	
}	

END

2.3.4.3 CPDLC Message ASN.1 Abstract Syntax

2.3.4.3.1 The abstract syntax of the CPDLC messages shall comply with the description contained in the ASN.1 module CPDLCMessageSetVersion1 conforming to ISO/IEC 8824, as defined in this section.

Note.— The object identifier “cpdlc-message-AS-v1” as defined below identifies the CPDLC Message Set Version 1:

cpdlc-message-AS-v1 ::= OBJECT IDENTIFIER {iso(1) identified organisation (3) icao(27) user message abstract syntax (10) cpdlc (1) version1 (1)}

CPDLCMessageSetVersion1 DEFINITIONS::=

BEGIN

ATCUplinkMessage ::= SEQUENCE

{	
header	ATCMessageHeader,
messageData	ATCUplinkMessageData
}	

ATCUplinkMessageData ::= SEQUENCE

{	
elementIds	SEQUENCE SIZE (1..5) OF ATCUplinkMsgElementId,
constrainedDataSEQUENCE	
{	
routeClearanceData	SEQUENCE SIZE (1..2) OF RouteClearance OPTIONAL,
...	
}	
OPTIONAL	
}	

ATCDownlinkMessage ::= SEQUENCE

```
{
  header          ATCMessageHeader,
  messageData     ATCDownlinkMessageData
}
```

ATCDownlinkMessageData ::= SEQUENCE

```
{
  elementIds      SEQUENCE SIZE (1..5) OF ATCDownlinkMsgElementId,
  constrainedData SEQUENCE
    {
      routeClearanceData SEQUENCE SIZE (1..2) OF RouteClearance OPTIONAL,
      ...
    }
  OPTIONAL
}
```

 -- Uplink and Downlink messages - Common Elements

ATCMessageHeader ::= SEQUENCE

```
{
  messageIdNumber      [0]  MsgIdentificationNumber,
  messageRefNumber     [1]  MsgReferenceNumber      OPTIONAL,
  dateTime              [2]  DateTimeGroup,
  logicalAck            [3]  LogicalAck              DEFAULT notRequired
}
```

MsgIdentificationNumber ::= INTEGER (0..63)

MsgReferenceNumber ::= INTEGER (0..63)

LogicalAck ::= ENUMERATED

```
{
  required              (0),
  notRequired           (1)
}
```

 -- Uplink message element

ATCUplinkMsgElementId ::= CHOICE

```
-- {
  UNABLE              Urg(N)/Alr(M)/Resp(N)
  uM0NULL             [0] NULL,
```

--	STANDBY uM1NULL	Urg(N)/Alr(L)/Resp(N) [1] NULL,
--	REQUEST DEFERRED uM2NULL	Urg(N)/Alr(L)/Resp(N) [2] NULL,
--	ROGER uM3NULL	Urg(N)/Alr(L)/Resp(N) [3] NULL,
--	AFFIRM uM4NULL	Urg(N)/Alr(L)/Resp(N) [4] NULL,
--	NEGATIVE uM5NULL	Urg(N)/Alr(L)/Resp(N) [5] NULL,
--	EXPECT [level] uM6Level	Urg(L)/Alr(L)/Resp(R) [6] Level,
--	EXPECT CLIMB AT [time] uM7Time	Urg(L)/Alr(L)/Resp(R) [7] Time,
--	EXPECT CLIMB AT [position] uM8Position	Urg(L)/Alr(L)/Resp(R) [8] Position,
--	EXPECT DESCENT AT [time] uM9Time	Urg(L)/Alr(L)/Resp(R) [9] Time,
--	EXPECT DESCENT AT [position] uM10Position	Urg(L)/Alr(L)/Resp(R) [10] Position,
--	EXPECT CRUISE CLIMB AT [time] uM11Time	Urg(L)/Alr(L)/Resp(R) [11] Time,
--	EXPECT CRUISE CLIMB AT [position] uM12Position	Urg(L)/Alr(L)/Resp(R) [12] Position,
--	AT [time] EXPECT CLIMB TO [level] uM13TimeLevel	Urg(L)/Alr(L)/Resp(R) [13] TimeLevel,
--	AT [position] EXPECT CLIMB TO [level] uM14PositionLevel	Urg(L)/Alr(L)/Resp(R) [14] PositionLevel,
--	AT [time] EXPECT DESCENT TO [level] uM15TimeLevel	Urg(L)/Alr(L)/Resp(R) [15] TimeLevel,
--	AT [position] EXPECT DESCENT TO [level] uM16PositionLevel	Urg(L)/Alr(L)/Resp(R) [16] PositionLevel,

--	AT [time] EXPECT CRUISE CLIMB TO [level] uM17TimeLevel	Urg(L)/Alr(L)/Resp(R) [17] TimeLevel,
--	AT [position] EXPECT CRUISE CLIMB TO [level] uM18PositionLevel	Urg(L)/Alr(L)/Resp(R) [18] PositionLevel,
--	MAINTAIN [level] uM19Level	Urg(N)/Alr(M)/Resp(W/U) [19] Level,
--	CLIMB TO [level] uM20Level	Urg(N)/Alr(M)/Resp(W/U) [20] Level,
--	AT [time] CLIMB TO [level] uM21TimeLevel	Urg(N)/Alr(M)/Resp(W/U) [21] TimeLevel,
--	AT [position] CLIMB TO [level] uM22PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [22] PositionLevel,
--	DESCEND TO [level] uM23Level	Urg(N)/Alr(M)/Resp(W/U) [23] Level,
--	AT [time] DESCEND TO [level] uM24TimeLevel	Urg(N)/Alr(M)/Resp(W/U) [24] TimeLevel,
--	AT [position] DESCEND TO [level] uM25PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [25] PositionLevel,
--	CLIMB TO REACH [level] BY [time] uM26LevelTime	Urg(N)/Alr(M)/Resp(W/U) [26] LevelTime,
--	CLIMB TO REACH [level] BY [position] uM27LevelPosition	Urg(N)/Alr(M)/Resp(W/U) [27] LevelPosition,
--	DESCEND TO REACH [level] BY [time] uM28LevelTime	Urg(N)/Alr(M)/Resp(W/U) [28] LevelTime,
--	DESCEND TO REACH [level] BY [position] uM29LevelPosition	Urg(N)/Alr(M)/Resp(W/U) [29] LevelPosition,
--	MAINTAIN BLOCK [level] TO [level] uM30LevelLevel	Urg(N)/Alr(M)/Resp(W/U) [30] LevelLevel,
--	CLIMB TO AND MAINTAIN BLOCK [level] TO [level] uM31LevelLevel	Urg(N)/Alr(M)/Resp(W/U) [31] LevelLevel,

--	DESCEND TO AND MAINTAIN BLOCK [level] TO [level]	
--	uM32LevelLevel	Urg(N)/Alr(M)/Resp(W/U) [32] LevelLevel,
--	Reserved	Urg(L)/Alr(L)/Resp(Y)
--	uM33NULL	[33] NULL,
--	CRUISE CLIMB TO [level]	Urg(N)/Alr(M)/Resp(W/U)
--	uM34Level	[34] Level,
--	CRUISE CLIMB ABOVE [level]	Urg(N)/Alr(M)/Resp(W/U)
--	uM35Level	[35] Level,
--	EXPEDITE CLIMB TO [level]	Urg(U)/Alr(M)/Resp(W/U)
--	uM36Level	[36] Level,
--	EXPEDITE DESCENT TO [level]	Urg(U)/Alr(M)/Resp(W/U)
--	uM37Level	[37] Level,
--	IMMEDIATELY CLIMB TO [level]	Urg(D)/Alr(H)/Resp(W/U)
--	uM38Level	[38] Level,
--	IMMEDIATELY DESCEND TO [level]	Urg(D)/Alr(H)/Resp(W/U)
--	uM39Level	[39] Level,
--	Reserved	Urg(L)/Alr(L)/Resp(Y)
--	uM40NULL	[40] NULL,
--	Reserved	Urg(L)/Alr(L)/Resp(Y)
--	uM41NULL	[41] NULL,
--	EXPECT TO CROSS [position] AT [level]	Urg(L)/Alr(L)/Resp(R)
--	uM42PositionLevel	[42] PositionLevel,
--	EXPECT TO CROSS [position] AT OR ABOVE [level]	
--	uM43PositionLevel	Urg(L)/Alr(L)/Resp(R) [43] PositionLevel,
--	EXPECT TO CROSS [position] AT OR BELOW [level]	
--	uM44PositionLevel	Urg(L)/Alr(L)/Resp(R) [44] PositionLevel,
--	EXPECT TO CROSS [position] AT AND MAINTAIN [level]	
--	uM45PositionLevel	Urg(L)/Alr(L)/Resp(R) [45] PositionLevel,

--	CROSS [position] AT [level] uM46PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [46] PositionLevel,
--	CROSS [position] AT OR ABOVE [level] uM47PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [47] PositionLevel,
--	CROSS [position] AT OR BELOW [level] uM48PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [48]PositionLevel,
--	CROSS [position] AT AND MAINTAIN [level] uM49PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [49] PositionLevel,
--	CROSS [position] BETWEEN [level] AND [level] uM50PositionLevelLevel	Urg(N)/Alr(M)/Resp(W/U) [50] PositionLevelLevel,
--	CROSS [position] AT [time] uM51PositionTime	Urg(N)/Alr(M)/Resp(W/U) [51] PositionTime,
--	CROSS [position] AT OR BEFORE [time] uM52PositionTime	Urg(N)/Alr(M)/Resp(W/U) [52] PositionTime,
--	CROSS [position] AT OR AFTER [time] uM53PositionTime	Urg(N)/Alr(M)/Resp(W/U) [53] PositionTime,
--	CROSS [position] BETWEEN [time] AND [time] uM54PositionTimeTime	Urg(N)/Alr(M)/Resp(W/U) [54] PositionTimeTime,
--	CROSS [position] AT [speed] uM55PositionSpeed	Urg(N)/Alr(M)/Resp(W/U) [55] PositionSpeed,
--	CROSS [position] AT OR LESS THAN [speed] uM56PositionSpeed	Urg(N)/Alr(M)/Resp(W/U) [56] PositionSpeed,
--	CROSS [position] AT OR GREATER THAN [speed] uM57PositionSpeed	Urg(N)/Alr(M)/Resp(W/U) [57] PositionSpeed,
--	CROSS [position] AT [time] AT [level] uM58PositionTimeLevel	Urg(N)/Alr(M)/Resp(W/U) [58] PositionTimeLevel,
--	CROSS [position] AT OR BEFORE [time] AT [level] uM59PositionTimeLevel	Urg(N)/Alr(M)/Resp(W/U) [59] PositionTimeLevel,
--	CROSS [position] AT OR AFTER [time] AT [level] uM60PositionTimeLevel	Urg(N)/Alr(M)/Resp(W/U) [60] PositionTimeLevel,

--	CROSS [position] AT AND MAINTAIN [level] AT [speed]	
--	uM61PositionLevelSpeed	Urg(N)/Alr(M)/Resp(W/U) [61] PositionLevelSpeed,
--	AT [time] CROSS [position] AT AND MAINTAIN [level]	
--	uM62TimePositionLevel	Urg(N)/Alr(M)/Resp(W/U) [62] TimePositionLevel,
--	AT [time] CROSS [position] AT AND MAINTAIN [level] AT [speed]	
--	uM63TimePositionLevelSpeed	Urg(N)/Alr(M)/Resp(W/U) [63] TimePositionLevelSpeed,
--	OFFSET [specifiedDistance] [direction] OF ROUTE	Urg(N)/Alr(M)/Resp(W/U)
--	uM64DistanceSpecifiedDirection	[64] DistanceSpecifiedDirection,
--	AT [position] OFFSET [specifiedDistance] [direction] OF ROUTE	
--	uM65PositionDistanceSpecifiedDirection	Urg(N)/Alr(M)/Resp(W/U) [65] PositionDistanceSpecifiedDirection,
--	AT [time] OFFSET [specifiedDistance] [direction] OF ROUTE	
--	uM66TimeDistanceSpecifiedDirection	Urg(N)/Alr(M)/Resp(W/U) [66] TimeDistanceSpecifiedDirection,
--	PROCEED BACK ON ROUTE	Urg(N)/Alr(M)/Resp(W/U)
--	uM67NULL	[67] NULL,
--	REJOIN ROUTE BY [position]	Urg(N)/Alr(M)/Resp(W/U)
--	uM68Position	[68] Position,
--	REJOIN ROUTE BY [time]	Urg(N)/Alr(M)/Resp(W/U)
--	uM69Time	[69] Time,
--	EXPECT BACK ON ROUTE BY [position]	Urg(L)/Alr(L)/Resp(R)
--	uM70Position	[70] Position,
--	EXPECT BACK ON ROUTE BY [time]	Urg(L)/Alr(L)/Resp(R)
--	uM71Time	[71] Time,
--	RESUME OWN NAVIGATION	Urg(N)/Alr(M)/Resp(W/U)
--	uM72NULL	[72] NULL,
--	[DepartureClearance]	Urg(N)/Alr(M)/Resp(W/U)
--	uM73DepartureClearance	[73] DepartureClearance,
--	PROCEED DIRECT TO [position]	Urg(N)/Alr(M)/Resp(W/U)
--	uM74Position	[74] Position,

--	WHEN ABLE PROCEED DIRECT TO [position] uM75Position	Urg(N)/Alr(M)/Resp(W/U) [75] Position,
--	AT [time] PROCEED DIRECT TO [position] uM76TimePosition	Urg(N)/Alr(M)/Resp(W/U) [76] TimePosition,
--	AT [position] PROCEED DIRECT TO [position] uM77PositionPosition	Urg(N)/Alr(M)/Resp(W/U) [77] PositionPosition,
--	AT [level] PROCEED DIRECT TO [position] uM78LevelPosition	Urg(N)/Alr(M)/Resp(W/U) [78] LevelPosition,
--	CLEARED TO [position] VIA [routeClearance] uM79PositionRouteClearance	Urg(N)/Alr(M)/Resp(W/U) [79] PositionRouteClearanceIndex,
--	CLEARED [routeClearance] uM80RouteClearance	Urg(N)/Alr(M)/Resp(W/U) [80] RouteClearanceIndex,
--	CLEARED [procedureName] uM81ProcedureName	Urg(N)/Alr(M)/Resp(W/U) [81] ProcedureName,
--	CLEARED TO DEVIATE UP TO [specifiedDistance] [direction] OF ROUTE uM82DistanceSpecifiedDirection	Urg(N)/Alr(M)/Resp(W/U) [82] DistanceSpecifiedDirection,
--	AT [position] CLEARED [routeClearance] uM83PositionRouteClearance	Urg(N)/Alr(M)/Resp(W/U) [83] PositionRouteClearanceIndex,
--	AT [position] CLEARED [procedureName] uM84PositionProcedureName	Urg(N)/Alr(M)/Resp(W/U) [84] PositionProcedureName,
--	EXPECT [routeClearance] uM85RouteClearance	Urg(L)/Alr(L)/Resp(R) [85] RouteClearanceIndex,
--	AT [position] EXPECT [routeClearance] uM86PositionRouteClearance	Urg(L)/Alr(L)/Resp(R) [86] PositionRouteClearanceIndex,
--	EXPECT DIRECT TO [position] uM87Position	Urg(L)/Alr(L)/Resp(R) [87] Position,
--	AT [position] EXPECT DIRECT TO [position] uM88PositionPosition	Urg(L)/Alr(L)/Resp(R) [88] PositionPosition,
--	AT [time] EXPECT DIRECT TO [position] uM89TimePosition	Urg(L)/Alr(L)/Resp(R) [89] TimePosition,
--	AT [level] EXPECT DIRECT TO [position] uM90LevelPosition	Urg(L)/Alr(L)/Resp(R) [90] LevelPosition,

--	HOLD AT [position] MAINTAIN [level] INBOUND TRACK [degrees][direction]	
--	TURNS [legtype]	Urg(N)/Alr(M)/Resp(W/U)
	uM91HoldClearance	[91] HoldClearance,
--	HOLD AT [position] AS PUBLISHED MAINTAIN [level]	
--	uM92PositionLevel	Urg(N)/Alr(M)/Resp(W/U)
		[92] PositionLevel,
--	EXPECT FURTHER CLEARANCE AT [time]	Urg(L)/Alr(L)/Resp(R)
	uM93Time	[93] Time,
--	TURN [direction] HEADING [degrees]	Urg(N)/Alr(M)/Resp(W/U)
	uM94DirectionDegrees	[94] DirectionDegrees,
--	TURN [direction] GROUND TRACK [degrees]	Urg(N)/Alr(M)/Resp(W/U)
	uM95DirectionDegrees	[95] DirectionDegrees,
--	CONTINUE PRESENT HEADING	Urg(N)/Alr(M)/Resp(W/U)
	uM96NULL	[96] NULL,
--	AT [position] FLY HEADING [degrees]	Urg(N)/Alr(M)/Resp(W/U)
	uM97PositionDegrees	[97] PositionDegrees,
--	IMMEDIATELY TURN [direction] HEADING [degrees]	
--	uM98DirectionDegrees	Urg(D)/Alr(H)/Resp(W/U)
		[98] DirectionDegrees,
--	EXPECT [procedureName]	Urg(L)/Alr(L)/Resp(R)
	uM99ProcedureName	[99] ProcedureName,
--	AT [time] EXPECT [speed]	Urg(L)/Alr(L)/Resp(R)
	uM100TimeSpeed	[100] TimeSpeed,
--	AT [position] EXPECT [speed]	Urg(L)/Alr(L)/Resp(R)
	uM101PositionSpeed	[101] PositionSpeed,
--	AT [level] EXPECT [speed]	Urg(L)/Alr(L)/Resp(R)
	uM102LevelSpeed	[102] LevelSpeed,
--	AT [time] EXPECT [speed] TO [speed]	Urg(L)/Alr(L)/Resp(R)
	uM103TimeSpeedSpeed	[103] TimeSpeedSpeed,
--	AT [position] EXPECT [speed] TO [speed]	Urg(L)/Alr(L)/Resp(R)
	uM104PositionSpeedSpeed	[104] PositionSpeedSpeed,
--	AT [level] EXPECT [speed] TO [speed]	Urg(L)/Alr(L)/Resp(R)
	uM105LevelSpeedSpeed	[105] LevelSpeedSpeed,

--	MAINTAIN [speed] uM106Speed	Urg(N)/Alr(M)/Resp(W/U) [106] Speed,
--	MAINTAIN PRESENT SPEED uM107NULL	Urg(N)/Alr(M)/Resp(W/U) [107] NULL,
--	MAINTAIN [speed] OR GREATER uM108Speed	Urg(N)/Alr(M)/Resp(W/U) [108] Speed,
--	MAINTAIN [speed] OR LESS uM109Speed	Urg(N)/Alr(M)/Resp(W/U) [109] Speed,
--	MAINTAIN [speed] TO [speed] uM110SpeedSpeed	Urg(N)/Alr(M)/Resp(W/U) [110] SpeedSpeed,
--	INCREASE SPEED TO [speed] uM111Speed	Urg(N)/Alr(M)/Resp(W/U) [111] Speed,
--	INCREASE SPEED TO [speed] OR GREATER uM112Speed	Urg(N)/Alr(M)/Resp(W/U) [112] Speed,
--	REDUCE SPEED TO [speed] uM113Speed	Urg(N)/Alr(M)/Resp(W/U) [113] Speed,
--	REDUCE SPEED TO [speed] OR LESS uM114Speed	Urg(N)/Alr(M)/Resp(W/U) [114] Speed,
--	DO NOT EXCEED [speed] uM115Speed	Urg(N)/Alr(M)/Resp(W/U) [115] Speed,
--	RESUME NORMAL SPEED uM116NULL	Urg(N)/Alr(M)/Resp(W/U) [116] NULL,
--	CONTACT [unitname] [frequency] uM117UnitNameFrequency	Urg(N)/Alr(M)/Resp(W/U) [117] UnitNameFrequency,
--	AT [position] CONTACT [unitname] [frequency] uM118PositionUnitNameFrequency	Urg(N)/Alr(M)/Resp(W/U) [118] PositionUnitNameFrequency,
--	AT [time] CONTACT [unitname] [frequency] uM119TimeUnitNameFrequency	Urg(N)/Alr(M)/Resp(W/U) [119] TimeUnitNameFrequency,
--	MONITOR [unitname] [frequency] uM120UnitNameFrequency	Urg(N)/Alr(M)/Resp(W/U) [120] UnitNameFrequency,
--	AT [position] MONITOR [unitname] [frequency] uM121PositionUnitNameFrequency	Urg(N)/Alr(M)/Resp(W/U) [121] PositionUnitNameFrequency,

--	AT [time] MONITOR [unitname] [frequency] uM122TimeUnitNameFrequency	Urg(N)/Alr(M)/Resp(W/U) [122] TimeUnitNameFrequency,
--	SQUAWK [code] uM123Code	Urg(N)/Alr(M)/Resp(W/U) [123] Code,
--	STOP SQUAWK uM124NULL	Urg(N)/Alr(M)/Resp(W/U) [124] NULL,
--	SQUAWK MODE CHARLIE uM125NULL	Urg(N)/Alr(M)/Resp(W/U) [125] NULL,
--	STOP SQUAWK MODE CHARLIE uM126NULL	Urg(N)/Alr(M)/Resp(W/U) [126] NULL,
--	REPORT BACK ON ROUTE uM127NULL	Urg(N)/Alr(L)/Resp(W/U) [127] NULL,
--	REPORT LEAVING [level] uM128Level	Urg(N)/Alr(L)/Resp(W/U) [128] Level,
--	REPORT MAINTAINING [level] uM129Level	Urg(N)/Alr(L)/Resp(W/U) [129] Level,
--	REPORT PASSING [position] uM130Position	Urg(N)/Alr(L)/Resp(W/U) [130] Position,
--	REPORT REMAINING FUEL AND PERSONS ON BOARD uM131NULL	Urg(U)/Alr(M)/Resp(Y) [131] NULL,
--	REPORT POSITION uM132NULL	Urg(N)/Alr(M)/Resp(Y) [132] NULL,
--	REPORT PRESENT LEVEL uM133NULL	Urg(N)/Alr(M)/Resp(Y) [133] NULL,
--	REPORT [speedtype] [speedtype] [speedtype]SPEED uM134SpeedTypeSpeedTypeSpeedType	Urg(N)/Alr(M)/Resp(Y) [134] SpeedTypeSpeedTypeSpeedType,
--	CONFIRM ASSIGNED LEVEL uM135NULL	Urg(N)/Alr(L)/Resp(Y) [135] NULL,
--	CONFIRM ASSIGNED SPEED uM136NULL	Urg(N)/Alr(L)/Resp(Y) [136] NULL,

--	CONFIRM ASSIGNED ROUTE uM137NULL	Urg(N)/Alr(L)/Resp(Y) [137] NULL,
--	CONFIRM TIME OVER REPORTED WAYPOINT uM138NULL	Urg(N)/Alr(L)/Resp(Y) [138] NULL,
--	CONFIRM REPORTED WAYPOINT uM139NULL	Urg(N)/Alr(L)/Resp(Y) [139] NULL,
--	CONFIRM NEXT WAYPOINT uM140NULL	Urg(N)/Alr(L)/Resp(Y) [140] NULL,
--	CONFIRM NEXT WAYPOINT ETA uM141NULL	Urg(N)/Alr(L)/Resp(Y) [141] NULL,
--	CONFIRM ENSUING WAYPOINT uM142NULL	Urg(N)/Alr(L)/Resp(Y) [142] NULL,
--	CONFIRM REQUEST uM143NULL	Urg(N)/Alr(L)/Resp(Y) [143] NULL,
--	CONFIRM SQUAWK uM144NULL	Urg(N)/Alr(L)/Resp(Y) [144] NULL,
--	REPORT HEADING uM145NULL	Urg(N)/Alr(M)/Resp(Y) [145] NULL,
--	REPORT GROUND TRACK uM146NULL	Urg(N)/Alr(M)/Resp(Y) [146] NULL,
--	REQUEST POSITION REPORT uM147NULL	Urg(N)/Alr(M)/Resp(Y) [147] NULL,
--	WHEN CAN YOU ACCEPT [level] uM148Level	Urg(N)/Alr(L)/Resp(Y) [148] Level,
--	CAN YOU ACCEPT [level] AT [position] uM149LevelPosition	Urg(N)/Alr(L)/Resp(A/N) [149] LevelPosition,
--	CAN YOU ACCEPT [level] AT [time] uM150LevelTime	Urg(N)/Alr(L)/Resp(A/N) [150] LevelTime,
--	WHEN CAN YOU ACCEPT [speed] uM151Speed	Urg(N)/Alr(L)/Resp(Y) [151] Speed,
--	WHEN CAN YOU ACCEPT [specifiedDistance] [direction] OFFSET	
--	uM152DistanceSpecifiedDirection	Urg(N)/Alr(L)/Resp(Y) [152] DistanceSpecifiedDirection,

--	ALTIMETER [altimeter] uM153Altimeter	Urg(N)/Alr(L)/Resp(R) [153] Altimeter,
--	RADAR SERVICE TERMINATED uM154NULL	Urg(N)/Alr(L)/Resp(R) [154] NULL,
--	RADAR CONTACT [position] uM155Position	Urg(N)/Alr(M)/Resp(R) [155] Position,
--	RADAR CONTACT LOST uM156NULL	Urg(N)/Alr(M)/Resp(R) [156] NULL,
--	CHECK STUCK MICROPHONE [frequency] uM157Frequency	Urg(U)/Alr(M)/Resp(N) [157] Frequency,
--	ATIS [atiscode] uM158AtisCode	Urg(N)/Alr(L)/Resp(R) [158] ATISCode,
--	ERROR [errorInformation] uM159ErrorInformation	Urg(U)/Alr(M)/Resp(N) [159] ErrorInformation,
--	NEXT DATA AUTHORITY [facility] uM160Facility	Urg(L)/Alr(N)/Resp(N) [160] Facility,
--	END SERVICE uM161NULL	Urg(L)/Alr(N)/Resp(N) [161] NULL,
--	SERVICE UNAVAILABLE uM162NULL	Urg(L)/Alr(L)/Resp(N) [162] NULL,
--	[facilitydesignation] uM163FacilityDesignation	Urg(L)/Alr(N)/Resp(N) [163] FacilityDesignation,
--	WHEN READY uM164NULL	Urg(L)/Alr(N)/Resp(N) [164] NULL,
--	THEN uM165NULL	Urg(L)/Alr(N)/Resp(N) [165] NULL,
--	DUE TO [traffictype]TRAFFIC uM166TrafficType	Urg(L)/Alr(N)/Resp(N) [166] TrafficType,
--	DUE TO AIRSPACE RESTRICTION uM167NULL	Urg(L)/Alr(N)/Resp(N) [167] NULL,
--	DISREGARD uM168NULL	Urg(U)/Alr(M)/Resp(R) [168] NULL,

--	[freetext] uM169FreeText	Urg(N)/Alr(L)/Resp(R) [169] FreeText,
--	[freetext] uM170FreeText	Urg(D)/Alr(H)/Resp(R) [170] FreeText,
--	CLIMB AT [verticalRate] MINIMUM uM171VerticalRate	Urg(N)/Alr(M)/Resp(W/U) [171] VerticalRate,
--	CLIMB AT [verticalRate] MAXIMUM uM172VerticalRate	Urg(N)/Alr(M)/Resp(W/U) [172] VerticalRate,
--	DESCEND AT [verticalRate] MINIMUM uM173VerticalRate	Urg(N)/Alr(M)/Resp(W/U) [173] VerticalRate,
--	DESCEND AT [verticalRate] MAXIMUM uM174VerticalRate	Urg(N)/Alr(M)/Resp(W/U) [174] VerticalRate,
--	REPORT REACHING [level] uM175Level	Urg(N)/Alr(L)/Resp(W/U) [175] Level,
--	MAINTAIN OWN SEPARATION AND VMC uM176NULL	Urg(N)/Alr(M)/Resp(W/U) [176] NULL,
--	AT PILOTS DISCRETION uM177NULL	Urg(L)/Alr(L)/Resp(N) [177] NULL,
--	Reserved uM178NULL	Urg(L)/Alr(L)/Resp(Y) [178] NULL,
--	SQUAWK IDENT uM179NULL	Urg(N)/Alr(M)/Resp(W/U) [179] NULL,
--	REPORT REACHING BLOCK [level] TO [level] uM180LevelLevel	Urg(N)/Alr(L)/Resp(W/U) [180] LevelLevel,
--	REPORT DISTANCE [tofrom] [position] uM181ToFromPosition	Urg(N)/Alr(M)/Resp(Y) [181] ToFromPosition,
--	CONFIRM ATIS CODE uM182NULL	Urg(N)/Alr(L)/Resp(Y) [182] NULL,
--	[freetext] uM183FreeText	Urg(N)/Alr(M)/Resp(N) [183] FreeText,
--	AT [time] REPORT DISTANCE [tofrom] [position] uM184TimeToFromPosition	Urg(N)/Alr(L)/Resp(Y) [184] TimeToFromPosition,

--	AFTER PASSING [position] CLIMB TO [level] uM185PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [185] PositionLevel,
--	AFTER PASSING [position] DESCEND TO [level] uM186PositionLevel	Urg(N)/Alr(M)/Resp(W/U) [186] PositionLevel,
--	[freetext] uM187FreeText	Urg(L)/Alr(N)/Resp(N) [187] FreeText,
--	AFTER PASSING [position] MAINTAIN [speed] uM188PositionSpeed	Urg(N)/Alr(M)/Resp(W/U) [188] PositionSpeed,
--	ADJUST SPEED TO [speed] uM189Speed	Urg(N)/Alr(M)/Resp(W/U) [189] Speed,
--	FLY HEADING [degrees] uM190Degrees	Urg(N)/Alr(M)/Resp(W/U) [190] Degrees,
--	ALL ATS TERMINATED uM191NULL	Urg(N)/Alr(M)/Resp(R) [191] NULL,
--	REACH [level] BY [time] uM192LevelTime	Urg(N)/Alr(M)/Resp(W/U) [192] LevelTime,
--	IDENTIFICATION LOST uM193NULL	Urg(N)/Alr(M)/Resp(R) [193] NULL,
--	[freetext] uM194FreeText	Urg(N)/Alr(L)/Resp(Y) [194] FreeText,
--	[freetext] uM195FreeText	Urg(L)/Alr(L)/Resp(R) [195] FreeText,
--	[freetext] uM196FreeText	Urg(N)/Alr(M)/Resp(W/U) [196] FreeText,
--	[freetext] uM197FreeText	Urg(U)/Alr(M)/Resp(W/U) [197] FreeText,
--	[freetext] uM198FreeText	Urg(D)/Alr(H)/Resp(W/U) [198] FreeText,
--	[freetext] uM199FreeText	Urg(N)/Alr(L)/Resp(N) [199] FreeText,
--	REPORT REACHING uM200NULL	Urg(N)/Alr(L)/Resp(W/U) [200] NULL,

--	Not Used uM201NULL	Urg(L)/Alr(L)/Resp(N) [201] NULL,
--	Not Used uM202NULL	Urg(L)/Alr(L)/Resp(N) [202] NULL,
--	[freetext] uM203FreeText	Urg(N)/Alr(M)/Resp(R) [203] FreeText,
--	[freetext] uM204FreeText	Urg(N)/Alr(M)/Resp(Y) [204] FreeText,
--	[freetext] uM205FreeText	Urg(N)/Alr(M)/Resp(A/N) [205] FreeText,
--	[freetext] uM206FreeText	Urg(L)/Alr(N)/Resp(Y) [206] FreeText,
--	[freetext] uM207FreeText	Urg(L)/Alr(L)/Resp(Y) [207] FreeText,
--	[freetext] uM208FreeText	Urg(L)/Alr(L)/Resp(N) [208] FreeText,
--	REACH [level] BY [position] uM209LevelPosition	Urg(N)/Alr(M)/Resp(W/U) [209] LevelPosition,
--	IDENTIFIED [position] uM210Position	Urg(N)/Alr(M)/Resp(R) [210] Position,
--	REQUEST FORWARDED uM211NULL	Urg(N)/Alr(L)/Resp(N) [211] NULL,
--	[facilitydesignation] ATIS [atiscode] CURRENT uM212FacilityDesignationATISCode	Urg(N)/Alr(L)/Resp(R) [212] FacilityDesignationATISCode,
--	[facilitydesignation] ALTIMETER [altimeter] uM213FacilityDesignationAltimeter	Urg(N)/Alr(L)/Resp(R) [213] FacilityDesignationAltimeter,
--	RVR RUNWAY [runway] [rvr] uM214RunwayRVR	Urg(N)/Alr(M)/Resp(R) [214] RunwayRVR,
--	TURN [direction][degrees] uM215DirectionDegrees	Urg(N)/Alr(M)/Resp(W/U) [215] DirectionDegrees,
--	REQUEST FLIGHT PLAN uM216NULL	Urg(N)/Alr(M)/Resp(Y) [216] NULL,

--	REPORT ARRIVAL uM217NULL	Urg(N)/Alr(M)/Resp(Y) [217] NULL,
--	REQUEST ALREADY RECEIVED uM218NULL	Urg(L)/Alr(N)/Resp(N) [218] NULL,
--	STOP CLIMB AT [level] uM219Level	Urg(U)/Alr(M)/Resp(W/U) [219] Level,
--	STOP DESCENT AT [level] uM220Level	Urg(U)/Alr(M)/Resp(W/U) [220] Level,
--	STOP TURN HEADING [degrees] uM221Degrees	Urg(U)/Alr(M)/Resp(W/U) [221] Degrees,
--	NO SPEED RESTRICTION uM222NULL	Urg(L)/Alr(L)/Resp(R) [222] NULL,
--	REDUCE TO MINIMUM APPROACH SPEED uM223NULL	Urg(N)/Alr(M)/Resp(W/U) [223] NULL,
--	NO DELAY EXPECTED uM224NULL	Urg(N)/Alr(L)/Resp(R) [224] NULL,
--	DELAY NOT DETERMINED uM225NULL	Urg(N)/Alr(L)/Resp(R) [225] NULL,
--	EXPECTED APPROACH TIME [time] uM226Time	Urg(N)/Alr(L)/Resp(R) [226] Time,
--	LOGICAL ACKNOWLEDGMENT uM227NULL	Urg(N)/Alr(M)/Resp(N) [227] NULL,
--	REPORT ETA [position] uM228Position	Urg(L)/Alr(L)/Resp(Y) [228] Position,
--	REPORT ALTERNATE AERODROME uM229NULL	Urg(L)/Alr(L)/Resp(Y) [229] NULL,
--	IMMEDIATELY uM230NULL	Urg(D)/Alr(H)/Resp(N) [230] NULL,
--	STATE PREFERRED LEVEL uM231NULL	Urg(L)/Alr(L)/Resp(Y) [231] NULL,
--	STATE TOP OF DESCENT uM232NULL	Urg(L)/Alr(L)/Resp(Y) [232] NULL,

```

-- USE OF LOGICAL ACKNOWLEDGMENT PROHIBITED
--                                     Urg(N)/Alr(M)/Resp(N)
    uM233NULL                        [233] NULL,

-- FLIGHT PLAN NOT HELD
--                                     Urg(L)/Alr(L)/Resp(N)
    uM234NULL                        [234] NULL,

-- ROGER 7500
--                                     Urg(U)/Alr(H)/Resp(N)
    uM235NULL                        [235] NULL,

-- LEAVE CONTROLLED AIRSPACE
--                                     Urg(N)/Alr(M)/Resp(W/U)
    uM236NULL                        [236] NULL,

-- REQUEST AGAIN WITH NEXT UNIT
--                                     Urg(N)/Alr(L)/Resp(N)
    uM237NULL                        [237] NULL,

    ...

    }

```

```

-----
-- Downlink message element
-----

```

ATCDownlinkMsgElementId ::= CHOICE

```

{
-- WILCO                                     Urg(N)/Alr(M)/Resp(N)
  dM0NULL                                  [0] NULL,

-- UNABLE                                     Urg(N)/Alr(M)/Resp(N)
  dM1NULL                                  [1] NULL,

-- STANDBY                                   Urg(N)/Alr(M)/Resp(N)
  dM2NULL                                  [2] NULL,

-- ROGER                                     Urg(N)/Alr(M)/Resp(N)
  dM3NULL                                  [3] NULL,

-- AFFIRM                                    Urg(N)/Alr(M)/Resp(N)
  dM4NULL                                  [4] NULL,

-- NEGATIVE                                  Urg(N)/Alr(M)/Resp(N)
  dM5NULL                                  [5] NULL,

-- REQUEST [level]                          Urg(N)/Alr(L)/Resp(Y)
  dM6Level                                [6] Level,

```

--	REQUEST BLOCK [level] TO [level] dM7LevelLevel	Urg(N)/Alr(L)/Resp(Y) [7] LevelLevel,
--	REQUEST CRUISE CLIMB TO [level] dM8Level	Urg(N)/Alr(L)/Resp(Y) [8] Level,
--	REQUEST CLIMB TO [level] dM9Level	Urg(N)/Alr(L)/Resp(Y) [9] Level,
--	REQUEST DESCENT TO [level] dM10Level	Urg(N)/Alr(L)/Resp(Y) [10] Level,
--	AT [position] REQUEST CLIMB TO [level] dM11PositionLevel	Urg(N)/Alr(L)/Resp(Y) [11] PositionLevel,
--	AT [position] REQUEST DESCENT TO [level] dM12PositionLevel	Urg(N)/Alr(L)/Resp(Y) [12] PositionLevel,
--	AT [time] REQUEST CLIMB TO [level] dM13TimeLevel	Urg(N)/Alr(L)/Resp(Y) [13] TimeLevel,
--	AT [time] REQUEST DESCENT TO [level] dM14TimeLevel	Urg(N)/Alr(L)/Resp(Y) [14] TimeLevel,
--	REQUEST OFFSET [specifiedDistance] [direction] OF ROUTE dM15DistanceSpecifiedDirection	Urg(N)/Alr(L)/Resp(Y) [15] DistanceSpecifiedDirection,
--	AT [position] REQUEST OFFSET [specifiedDistance] [direction] OF ROUTE dM16PositionDistanceSpecifiedDirection	Urg(N)/Alr(L)/Resp(Y) [16] PositionDistanceSpecifiedDirection,
--	AT [time] REQUEST OFFSET [specifiedDistance] [direction] OF ROUTE dM17TimeDistanceSpecifiedDirection	Urg(N)/Alr(L)/Resp(Y) [17] TimeDistanceSpecifiedDirection,
--	REQUEST [speed] dM18Speed	Urg(N)/Alr(L)/Resp(Y) [18] Speed,
--	REQUEST [speed] TO [speed] dM19SpeedSpeed	Urg(N)/Alr(L)/Resp(Y) [19] SpeedSpeed,
--	REQUEST VOICE CONTACT dM20NULL	Urg(N)/Alr(L)/Resp(Y) [20] NULL,
--	REQUEST VOICE CONTACT [frequency] dM21Frequency	Urg(N)/Alr(L)/Resp(Y) [21] Frequency,

--	REQUEST DIRECT TO [position] dM22Position	Urg(N)/Alr(L)/Resp(Y) [22] Position,
--	REQUEST [procedureName] dM23ProcedureName	Urg(N)/Alr(L)/Resp(Y) [23] ProcedureName,
--	REQUEST CLEARANCE [routeClearance] dM24RouteClearance	Urg(N)/Alr(L)/Resp(Y) [24] RouteClearanceIndex,
--	REQUEST [clearanceType] CLEARANCE dM25ClearanceType	Urg(N)/Alr(L)/Resp(Y) [25] ClearanceType,
--	REQUEST WEATHER DEVIATION TO [position] VIA [routeClearance] dM26PositionRouteClearance	Urg(N)/Alr(M)/Resp(Y) [26] PositionRouteClearanceIndex,
--	REQUEST WEATHER DEVIATION UP TO [specifiedDistance] [direction] OF ROUTE dM27DistanceSpecifiedDirection	Urg(N)/Alr(M)/Resp(Y) [27] DistanceSpecifiedDirection,
--	LEAVING [level] dM28Level	Urg(N)/Alr(L)/Resp(N) [28] Level,
--	CLIMBING TO [level] dM29Level	Urg(N)/Alr(L)/Resp(N) [29] Level,
--	DESCENDING TO [level] dM30Level	Urg(N)/Alr(L)/Resp(N) [30] Level,
--	PASSING [position] dM31Position	Urg(N)/Alr(L)/Resp(N) [31] Position,
--	PRESENT LEVEL [level] dM32Level	Urg(N)/Alr(L)/Resp(N) [32] Level,
--	PRESENT POSITION [position] dM33Position	Urg(N)/Alr(L)/Resp(N) [33] Position,
--	PRESENT SPEED [speed] dM34Speed	Urg(N)/Alr(L)/Resp(N) [34] Speed,
--	PRESENT HEADING [degrees] dM35Degrees	Urg(N)/Alr(L)/Resp(N) [35] Degrees,
--	PRESENT GROUND TRACK [degrees] dM36Degrees	Urg(N)/Alr(L)/Resp(N) [36] Degrees,

--	MAINTAINING [level] dM37Level	Urg(N)/Alr(L)/Resp(N) [37] Level,
--	ASSIGNED LEVEL [level] dM38Level	Urg(N)/Alr(M)/Resp(N) [38] Level,
--	ASSIGNED SPEED [speed] dM39Speed	Urg(N)/Alr(M)/Resp(N) [39] Speed,
--	ASSIGNED ROUTE [routeClearance] dM40RouteClearance	Urg(N)/Alr(M)/Resp(N) [40] RouteClearanceIndex,
--	BACK ON ROUTE dM41NULL	Urg(N)/Alr(M)/Resp(N) [41] NULL,
--	NEXT WAYPOINT [position] dM42Position	Urg(N)/Alr(L)/Resp(N) [42] Position,
--	NEXT WAYPOINT ETA [time] dM43Time	Urg(N)/Alr(L)/Resp(N) [43] Time,
--	ENSUING WAYPOINT [position] dM44Position	Urg(N)/Alr(L)/Resp(N) [44] Position,
--	REPORTED WAYPOINT [position] dM45Position	Urg(N)/Alr(L)/Resp(N) [45] Position,
--	REPORTED WAYPOINT [time] dM46Time	Urg(N)/Alr(L)/Resp(N) [46] Time,
--	SQUAWKING [code] dM47Code	Urg(N)/Alr(L)/Resp(N) [47] Code,
--	POSITION REPORT [positionreport] dM48PositionReport	Urg(N)/Alr(M)/Resp(N) [48] PositionReport,
--	WHEN CAN WE EXPECT [speed] dM49Speed	Urg(L)/Alr(L)/Resp(Y) [49] Speed,
--	WHEN CAN WE EXPECT [speed] TO [speed] dM50SpeedSpeed	Urg(L)/Alr(L)/Resp(Y) [50] SpeedSpeed,
--	WHEN CAN WE EXPECT BACK ON ROUTE dM51NULL	Urg(L)/Alr(L)/Resp(Y) [51] NULL,
--	WHEN CAN WE EXPECT LOWER LEVEL dM52NULL	Urg(L)/Alr(L)/Resp(Y) [52] NULL,

--	WHEN CAN WE EXPECT HIGHER LEVEL dM53NULL	Urg(L)/Alr(L)/Resp(Y) [53] NULL,
--	WHEN CAN WE EXPECT CRUISE CLIMB TO [level] dM54Level	Urg(L)/Alr(L)/Resp(Y) [54] Level,
--	PAN PAN PAN dM55NULL	Urg(U)/Alr(H)/Resp(Y) [55] NULL,
--	MAYDAY MAYDAY MAYDAY dM56NULL	Urg(D)/Alr(H)/Resp(Y) [56] NULL,
--	[remainingFuel] OF FUEL REMAINING AND [personsonboard] PERSONS ON BOARD dM57RemainingFuelPersonsOnBoard	Urg(U)/Alr(H)/Resp(Y) [57] RemainingFuelPersonsOnBoard,
--	CANCEL EMERGENCY dM58NULL	Urg(U)/Alr(M)/Resp(Y) [58] NULL,
--	DIVERTING TO [position] VIA [routeClearance] dM59PositionRouteClearance	Urg(U)/Alr(H)/Resp(Y) [59] PositionRouteClearanceIndex,
--	OFFSETTING [specifiedDistance] [direction] OF ROUTE dM60DistanceSpecifiedDirection	Urg(U)/Alr(H)/Resp(Y) [60] DistanceSpecifiedDirection,
--	DESCENDING TO [level] dM61Level	Urg(U)/Alr(H)/Resp(Y) [61] Level,
--	ERROR [errorInformation] dM62ErrorInformation	Urg(U)/Alr(L)/Resp(N) [62] ErrorInformation,
--	NOT CURRENT DATA AUTHORITY dM63NULL	Urg(L)/Alr(L)/Resp(N) [63] NULL,
--	[facilitydesignation] dM64FacilityDesignation	Urg(L)/Alr(L)/Resp(N) [64] FacilityDesignation,
--	DUE TO WEATHER dM65NULL	Urg(L)/Alr(L)/Resp(N) [65] NULL,
--	DUE TO AIRCRAFT PERFORMANCE dM66NULL	Urg(L)/Alr(L)/Resp(N) [66] NULL,
--	[freetext] dM67FreeText	Urg(N)/Alr(L)/Resp(N) [67] FreeText,

--	[freetext] dM68FreeText	Urg(D)/Alr(H)/Resp(Y) [68] FreeText,
--	REQUEST VMC DESCENT dM69NULL	Urg(N)/Alr(L)/Resp(Y) [69] NULL,
--	REQUEST HEADING [degrees] dM70Degrees	Urg(N)/Alr(L)/Resp(Y) [70] Degrees,
--	REQUEST GROUND TRACK [degrees] dM71Degrees	Urg(N)/Alr(L)/Resp(Y) [71] Degrees,
--	REACHING [level] dM72Level	Urg(N)/Alr(L)/Resp(N) [72] Level,
--	[versionnumber] dM73Versionnumber	Urg(L)/Alr(L)/Resp(N) [73] VersionNumber,
--	REQUEST TO MAINTAIN OWN SEPARATION AND VMC	
--	dM74NULL	Urg(L)/Alr(L)/Resp(Y) [74] NULL,
--	AT PILOTS DISCRETION dM75NULL	Urg(L)/Alr(L)/Resp(N) [75] NULL,
--	REACHING BLOCK [level] TO [level] dM76LevelLevel	Urg(N)/Alr(L)/Resp(N) [76] LevelLevel,
--	ASSIGNED BLOCK [level] TO [level] dM77LevelLevel	Urg(N)/Alr(M)/Resp(N) [77] LevelLevel,
--	AT [time] [distance] [tofrom] [position] dM78TimeDistanceToFromPosition	Urg(N)/Alr(L)/Resp(N) [78] TimeDistanceToFromPosition,
--	ATIS [atiscode] dM79AtisCode	Urg(N)/Alr(L)/Resp(N) [79] ATISCode,
--	DEVIATING UP TO [specifiedDistance] [direction] OF ROUTE	
--	dM80DistanceSpecifiedDirection	Urg(U)/Alr(H)/Resp(Y) [80] DistanceSpecifiedDirection,
--	WE CAN ACCEPT [level] AT [time] dM81LevelTime	Urg(L)/Alr(L)/Resp(N) [81] LevelTime,
--	WE CANNOT ACCEPT [level] dM82Level	Urg(L)/Alr(L)/Resp(N) [82] Level,

--	WE CAN ACCEPT [speed] AT [time] dM83SpeedTime	Urg(L)/Alr(L)/Resp(N) [83] SpeedTime,
--	WE CANNOT ACCEPT [speed] dM84Speed	Urg(L)/Alr(L)/Resp(N) [84] Speed,
--	WE CAN ACCEPT [specifiedDistance] [direction] AT [time] dM85DistanceSpecifiedDirectionTime	Urg(L)/Alr(L)/Resp(N) [85] DistanceSpecifiedDirectionTime,
--	WE CANNOT ACCEPT [specifiedDistance] [direction] dM86DistanceSpecifiedDirection	Urg(L)/Alr(L)/Resp(N) [86] DistanceSpecifiedDirection,
--	WHEN CAN WE EXPECT CLIMB TO [level] dM87Level	Urg(L)/Alr(L)/Resp(Y) [87] Level,
--	WHEN CAN WE EXPECT DESCENT TO [level] dM88Level	Urg(L)/Alr(L)/Resp(Y) [88] Level,
--	MONITORING [unitname] [frequency] dM89UnitnameFrequency	Urg(U)/Alr(M)/Resp(N) [89] UnitNameFrequency,
--	[freetext] dM90FreeText	Urg(N)/Alr(M)/Resp(N) [90] FreeText,
--	[freetext] dM91FreeText	Urg(N)/Alr(L)/Resp(Y) [91] FreeText,
--	[freetext] dM92FreeText	Urg(L)/Alr(L)/Resp(Y) [92] FreeText,
--	[freetext] dM93FreeText	Urg(U)/Alr(H)/Resp(N) [93] FreeText,
--	[freetext] dM94FreeText	Urg(D)/Alr(H)/Resp(N) [94] FreeText,
--	[freetext] dM95FreeText	Urg(U)/Alr(M)/Resp(N) [95] FreeText,
--	[freetext] dM96FreeText	Urg(U)/Alr(L)/Resp(N) [96] FreeText,
--	[freetext] dM97FreeText	Urg(L)/Alr(L)/Resp(N) [97] FreeText,

--	[freetext] dM98FreeText	Urg(N)/Alr(N)/Resp(N) [98] FreeText,
--	CURRENT DATA AUTHORITY dM99NULL	Urg(L)/Alr(L)/Resp(N) [99] NULL,
--	LOGICAL ACKNOWLEDGMENT dM100NULL	Urg(N)/Alr(M)/Resp(N) [100] NULL,
--	REQUEST END OF SERVICE dM101NULL	Urg(L)/Alr(L)/Resp(Y) [101] NULL,
--	LANDING REPORT dM102NULL	Urg(N)/Alr(N)/Resp(N) [102] NULL,
--	CANCELLING IFR dM103NULL	Urg(N)/Alr(L)/Resp(Y) [103] NULL,
--	ETA[position][time] dM104PositionTime	Urg(L)/Alr(L)/Resp(N) [104] PositionTime,
--	ALTERNATE AERODROME[airport] dM105Airport	Urg(L)/Alr(L)/Resp(N) [105] Airport,
--	PREFERRED LEVEL[level] dM106Level	Urg(L)/Alr(L)/Resp(N) [106] Level,
--	NOT AUTHORIZED NEXT DATA AUTHORITY dM107NULL	Urg(L)/Alr(L)/Resp(N) [107] NULL,
--	DE-ICING COMPLETE dM108NULL	Urg(L)/Alr(L)/Resp(N) [108] NULL,
--	TOP OF DESCENT [time] dM109Time	Urg(L)/Alr(L)/Resp(N) [109] Time,
--	TOP OF DESCENT [position] dM110Position	Urg(L)/Alr(L)/Resp(N) [110] Position,
--	TOP OF DESCENT [time] [position] dM111TimePosition	Urg(L)/Alr(L)/Resp(N) [111] TimePosition,
--	SQUAWKING 7500 dM112NULL	Urg(U)/Alr(H)/Resp(N) [112] NULL,

```
-- [speedType] [speedType] [speedType] SPEED [speed]      Urg(N)/Alr(L)/Resp(N)
   dm113SpeedTypeSpeedTypeSpeedTypeSpeed                [113]SpeedTypeSpeedTypeSpeedType
Speed,
...
}
```

AircraftAddress ::= BIT STRING (SIZE(24))

AircraftFlightIdentification ::= IA5String (SIZE (2..8))

Airport ::= IA5String (SIZE (4))

Altimeter ::= CHOICE

```
{
  altimeterEnglish      [0]    AltimeterEnglish,
  altimeterMetric       [1]    AltimeterMetric
}
```

AltimeterEnglish ::= INTEGER (2200..3200)

-- unit = Inches Mercury, Range (22.00 .. 32.00), resolution = 0.01

AltimeterMetric ::= INTEGER (7500..12500)

-- unit = Hectopascal, Range (750.0..1250.0), resolution = 0.1

ATISCode ::= IA5String (SIZE (1))

ATSRouteDesignator ::= IA5String (SIZE (2..7))

ATWAlongTrackWaypoint ::= SEQUENCE

```
{
  position              [0]    Position,
  aTWDistance           [1]    ATWDistance,
  speed                [2]    Speed                      OPTIONAL,
  aTWLevels             [3]    ATWLevelSequence          OPTIONAL
}
```

ATWLevel ::= SEQUENCE

```
{
  atw                  ATWLevelTolerance,
  level                Level
}
```

ATWLevelSequence ::= SEQUENCE SIZE (1..2) OF ATWLevel

ATWLevelTolerance ::= ENUMERATED

```
{  
  at (0),  
  atorabove (1),  
  atorbelow (2)  
}
```

ATWDistance ::= SEQUENCE

```
{  
  atwDistanceTolerance ATWDistanceTolerance,  
  distance Distance  
}
```

ATWDistanceTolerance ::= ENUMERATED

```
{  
  plus (0),  
  minus (1)  
}
```

ClearanceType ::= ENUMERATED

```
{  
  noneSpecified (0),  
  approach (1),  
  departure (2),  
  further (3),  
  start-up (4),  
  pushback (5),  
  taxi (6),  
  take-off (7),  
  landing (8),  
  oceanic (9),  
  en-route (10),  
  downstream (11),  
  ...  
}
```

Code ::= SEQUENCE SIZE (4) OF CodeOctalDigit

CodeOctalDigit ::= INTEGER (0..7)

ControlledTime ::= SEQUENCE

```
{  
  time Time,  
  timeTolerance TimeTolerance  
}
```

Date ::= SEQUENCE

```
{
  year          Year,
  month         Month,
  day           Day
}
```

DateTimeGroup ::= SEQUENCE

```
{
  date          Date,
  timehhmmss    Timehhmmss
}
```

Day ::= INTEGER (1..31)

--unit = Day, Range (1..31), resolution = 1

DegreeIncrement ::= INTEGER (1..20)

--unit = Degree, Range (1..20), resolution = 1

Degrees ::= CHOICE

```
{
  degreesMagnetic [0]    DegreesMagnetic,
  degreesTrue     [1]    DegreesTrue
}
```

DegreesMagnetic ::= INTEGER (1..360)

--unit = degree, Range (1..360), resolution = 1

DegreesTrue ::= INTEGER (1..360)

--unit = degree, Range (1..360), resolution = 1

DepartureClearance ::= SEQUENCE

```
{
  aircraftFlightIdentification[0]    AircraftFlightIdentification,
  clearanceLimit                    [1]    Position,
  flightInformation                  [2]    FlightInformation    OPTIONAL,
  furtherInstructions                [3]    FurtherInstructions    OPTIONAL
}
```

DepartureMinimumInterval ::= INTEGER (1..150)

--unit = Minute, Range (0.1..15.0), resolution = 0.1

Direction ::= ENUMERATED

```
{
  left           (0),
  right          (1),
  eitherSide     (2),
  north          (3),
  south          (4),
  east           (5),
  west           (6),
  northEast      (7),
  northWest      (8),
  southEast      (9),
  southWest      (10)
}
```

DirectionDegrees ::= SEQUENCE

```
{
  direction      Direction,
  degrees        Degrees
}
```

Distance ::= CHOICE

```
{
  distanceNm     [0]    DistanceNm,
  distanceKm     [1]    DistanceKm
}
```

DistanceKm ::= INTEGER (0..8000)

-- unit = Kilometer, Range (0..2000), resolution = 0.25

DistanceNm ::= INTEGER (0..9999)

-- unit = Nautical Mile, Range (0..999.9), resolution = 0.1

DistanceSpecified ::= CHOICE

```
{
  distanceSpecifiedNm [0]    DistanceSpecifiedNm,
  distanceSpecifiedKm [1]    DistanceSpecifiedKm
}
```

DistanceSpecifiedDirection ::= SEQUENCE

```
{
  distanceSpecified DistanceSpecified,
  direction          Direction
}
```

DistanceSpecifiedDirectionTime ::= SEQUENCE

```
{
  distanceSpecifiedDirection    DistanceSpecifiedDirection,
  time                          Time
}
```

DistanceSpecifiedKm ::= INTEGER (1..500)

-- unit = Kilometer, Range (1..500), resolution = 1

DistanceSpecifiedNm ::= INTEGER (1..250)

-- unit = Nautical Mile, Range (1..250), resolution = 1

ErrorInformation ::= ENUMERATED

```
{
  unrecognizedMsgReferenceNumber    (0),
  logicalAcknowledgmentNotAccepted  (1),
  insufficientResources              (2),
  invalidMessageElementCombination  (3),
  invalidMessageElement              (4),
  ...
}
```

Facility ::= CHOICE

```
{
  noFacility      [0]    NULL,
  facilityDesignation [1] FacilityDesignation
}
```

FacilityDesignation ::= IA5String (SIZE (4..8))

FacilityFunction ::= ENUMERATED

```
{
  center          (0),
  approach        (1),
  tower           (2),
  final           (3),
  groundControl   (4),
  clearanceDelivery (5),
  departure       (6),
  control         (7),
  radio           (8),
  ...
}
```

FacilityDesignationAltimeter ::= SEQUENCE

```
{
  facilityDesignation      FacilityDesignation,
  altimeter                Altimeter
}
```

FacilityDesignationATISCode ::= SEQUENCE

```
{
  facilityDesignation      FacilityDesignation,
  aTISCode                 ATISCode
}
```

FacilityName ::= IA5String (SIZE (3..18))

Fix ::= IA5String (SIZE (1..5))

FixName ::= SEQUENCE

```
{
  name                    [0]    Fix,
  latlon                  [1]    LatitudeLongitude    OPTIONAL
}
```

FlightInformation ::= CHOICE

```
{
  routeOfFlight           [0]    RouteInformation,
  levelsOfFlight           [1]    LevelsOfFlight,
  routeAndLevels           [2]    RouteAndLevels
}
```

FreeText ::= IA5String (SIZE (1..256))

Frequency ::= CHOICE

```
{
  frequencyhf             [0]    Frequencyhf,
  frequencyvhf            [1]    Frequencyvhf,
  frequencyuhf            [2]    Frequencyuhf,
  frequencysatchannel     [3]    Frequencysatchannel
}
```

Frequencyhf ::= INTEGER (2850..28000)

-- unit = KiloHertz, Range (2850..28000), resolution = 1

Frequencysatchannel ::= NumericString (SIZE (12))

-- Frequencysatchannel corresponds to a 12 digit telephone number

Frequencyuhf ::= INTEGER (9000..15999)

-- unit = Megahertz, Range (225.000..399.975), resolution = 0.025

Frequencyvhf ::= INTEGER (23600..27398)

-- unit = Megahertz, Range (118.000..136.990), resolution = 0.005

FurtherInstructions ::= SEQUENCE

{			
code	[0]	Code	OPTIONAL,
frequencyDeparture	[1]	UnitNameFrequency	OPTIONAL,
clearanceExpiryTime	[2]	Time	OPTIONAL,
airportDeparture	[3]	Airport	OPTIONAL,
airportDestination	[4]	Airport	OPTIONAL,
timeDeparture	[5]	TimeDeparture	OPTIONAL,
runwayDeparture	[6]	Runway	OPTIONAL,
revisionNumber	[7]	RevisionNumber	OPTIONAL,
aTISCode	[8]	ATISCode	OPTIONAL
}			

Holdatwaypoint ::= SEQUENCE

{			
position	[0]	Position,	
holdatwaypointspeedlow	[1]	Speed	OPTIONAL,
aTWlevel	[2]	ATWLevel	OPTIONAL,
holdatwaypointspeedhigh	[3]	Speed	OPTIONAL,
direction	[4]	Direction	OPTIONAL,
degrees	[5]	Degrees	OPTIONAL,
eFctime	[6]	Time	OPTIONAL,
legtype	[7]	LegType	OPTIONAL
}			

HoldClearance ::= SEQUENCE

{			
position	[0]	Position,	
level	[1]	Level,	
degrees	[2]	Degrees,	
direction	[3]	Direction,	
legType	[4]	LegType	OPTIONAL
}			

Humidity ::= INTEGER (0..100)

-- unit = Percent humidity, Range (0..100), resolution = 1

InterceptCourseFrom ::= SEQUENCE

{		
fromSelection	InterceptCourseFromSelection,	
degrees	Degrees	
}		

InterceptCourseFromSelection ::= CHOICE

{		
publishedIdentifier	[0]	PublishedIdentifier,
latitudeLongitude	[1]	LatitudeLongitude,
placeBearingPlaceBearing	[2]	PlaceBearingPlaceBearing,
placeBearingDistance	[3]	PlaceBearingDistance
}		

Icing ::= ENUMERATED

{	
reserved	(0),
light	(1),
moderate	(2),
severe	(3)
}	

Latitude ::= SEQUENCE

{	
latitudeType	LatitudeType,
latitudeDirection	LatitudeDirection
}	

LatitudeDegrees ::= INTEGER (0..90000)

-- unit = Degree, Range (0..90), resolution = 0.001

LatitudeDegreesMinutes ::= SEQUENCE

{	
latitudeWholeDegrees	LatitudeWholeDegrees,
minutesLatLon	MinutesLatLon
}	

LatitudeDegreesMinutesSeconds ::= SEQUENCE

{	
latitudeWholeDegrees	LatitudeWholeDegrees,
latlonWholeMinutes	LatLonWholeMinutes,
secondsLatLon	SecondsLatLon
}	

LatitudeDirection ::= ENUMERATED

{	
north	(0),
south	(1)
}	

LatitudeWholeDegrees ::= INTEGER (0..89)

-- unit = Degree, Range (0..89), resolution = 1

LatitudeLongitude ::= SEQUENCE

```
{
  latitude      [0]    Latitude    OPTIONAL,
  longitude     [1]    Longitude   OPTIONAL
}
```

LatitudeReportingPoints ::= SEQUENCE

```
{
  latitudeDirection    LatitudeDirection,
  latitudeDegrees      LatitudeDegrees
}
```

LatitudeType ::= CHOICE

```
{
  latitudeDegrees      [0]    LatitudeDegrees,
  latitudeDegreesMinutes [1]    LatitudeDegreesMinutes,
  latitudeDMS          [2]    LatitudeDegreesMinutesSeconds
}
```

LatLonWholeMinutes ::= INTEGER (0..59)

-- unit = Minute, Range (0..59), resolution = 1

LatLonReportingPoints ::= CHOICE

```
{
  latitudeReportingPoints [0]    LatitudeReportingPoints,
  longitudeReportingPoints [1]    LongitudeReportingPoints
}
```

LegDistance ::= CHOICE

```
{
  legDistanceEnglish      [0]    LegDistanceEnglish,
  legDistanceMetric       [1]    LegDistanceMetric
}
```

LegDistanceEnglish ::= INTEGER (0..50)

-- unit = Nautical Mile, Range (0..50), resolution = 1

LegDistanceMetric ::= INTEGER (1..128)

-- unit = Kilometer, Range (1..128), resolution = 1

LegTime ::= INTEGER (0..10)

--unit = Minute, Range (0..10), resolution = 1

LegType ::= CHOICE

```
{
  legDistance [0]    LegDistance,
  legTime     [1]    LegTime
}
```

Level ::= CHOICE

```
{
  singleLevel      [0]    LevelType,
  blockLevel       [1]    SEQUENCE SIZE (2) OF LevelType
}
```

LevelFeet ::= INTEGER (-60..7000)

--unit = Feet, Range (-600..70000), resolution = 10

LevelFlightLevel ::= INTEGER (30..700)

--unit = Level (100 Feet), Range (030..700), resolution = 1

LevelFlightLevelMetric ::= INTEGER (100..2500)

--unit = Level (10 Meters), Range (100..2500), resolution = 1

LevelLevel ::= SEQUENCE SIZE (2) OF Level

LevelMeters ::= INTEGER (-30..25000)

--unit = Meter, Range (-30..25000), resolution = 1

LevelPosition ::= SEQUENCE

```
{
  level              Level,
  position            Position
}
```

LevelProcedureName ::= SEQUENCE

```
{
  level              Level,
  procedureName      ProcedureName
}
```

LevelsOfFlight ::= CHOICE

```
{
  level              [0]    Level,
  procedureName      [1]    ProcedureName,
  levelProcedureName [2]    LevelProcedureName
}
```

LevelSpeed ::= SEQUENCE

```
{
  level      Level,
  speed      Speed
}
```

LevelSpeedSpeed ::= SEQUENCE

```
{
  level           Level,
  speeds          SpeedSpeed
}
```

LevelTime ::= SEQUENCE

```
{
  level           Level,
  time            Time
}
```

LevelType ::= CHOICE

```
{
  levelFeet       [0]    LevelFeet,
  levelMeters     [1]    LevelMeters,
  levelFlightLevel [2]    LevelFlightLevel,
  levelFlightLevelMetric [3] LevelFlightLevelMetric
}
```

Longitude ::= SEQUENCE

```
{
  longitudeType      LongitudeType,
  longitudeDirection LongitudeDirection
}
```

LongitudeDegrees ::= INTEGER (0..180000)

--unit = Degree, Range (0..180), resolution = 0.001

LongitudeDegreesMinutes ::= SEQUENCE

```
{
  longitudeWholeDegrees LongitudeWholeDegrees,
  minutesLatLon          MinutesLatLon
}
```

LongitudeDegreesMinutesSeconds ::= SEQUENCE

```
{
  longitudeWholeDegrees LongitudeWholeDegrees,
  latLonWholeMinutes    LatLonWholeMinutes,
  secondsLatLon         SecondsLatLon
}
```

LongitudeDirection ::= ENUMERATED

```
{
  east      (0),
  west      (1)
}
```

LongitudeWholeDegrees ::= INTEGER (0..179)

-- unit = Degree, Range (0..179), resolution = 1

LongitudeReportingPoints ::= SEQUENCE

```
{
  longitudeDirection      LongitudeDirection,
  longitudeDegrees        LongitudeDegrees
}
```

LongitudeType ::= CHOICE

```
{
  longitudeDegrees        [0]    LongitudeDegrees,
  longitudeDegreesMinutes [1]    LongitudeDegreesMinutes,
  longitudeDMS            [2]    LongitudeDegreesMinutesSeconds
}
```

MinutesLatLon ::= INTEGER (0..5999)

--unit = Minute, Range (0..59.99), resolution = 0.01

Month ::= INTEGER (1..12)

--unit = 1 Month, Range (1..12), resolution = 1

Navaid ::= SEQUENCE

```
{
  name          [0]    NavaidName,
  latlon        [1]    LatitudeLongitude    OPTIONAL
}
```

NavaidName ::= IA5String (SIZE (1..4))

PersonsOnBoard ::= INTEGER (1..1024)

PlaceBearing ::= SEQUENCE

```
{
  publishedIdentifier      PublishedIdentifier,
  degrees                  Degrees
}
```

PlaceBearingDistance ::= SEQUENCE

```
{
  publishedIdentifier      PublishedIdentifier,
  degrees                  Degrees,
  distance                  Distance
}
```

PlaceBearingPlaceBearing ::= SEQUENCE SIZE (2) OF PlaceBearing

Position ::= CHOICE

{		
fixName	[0]	FixName,
navaid	[1]	Navaid,
airport	[2]	Airport,
latitudeLongitude	[3]	LatitudeLongitude,
placeBearingDistance	[4]	PlaceBearingDistance
}		

PositionDegrees ::= SEQUENCE

{		
position		Position,
degrees		Degrees
}		

PositionDistanceSpecifiedDirection ::= SEQUENCE

{		
position		Position,
distanceSpecifiedDirection		DistanceSpecifiedDirection
}		

PositionLevel ::= SEQUENCE

{		
position		Position,
level		Level
}		

PositionLevelLevel ::= SEQUENCE

{		
position		Position,
levels		LevelLevel
}		

PositionLevelSpeed ::= SEQUENCE

{		
positionlevel		PositionLevel,
speed		Speed
}		

PositionPosition ::= SEQUENCE SIZE (2) OF Position

PositionProcedureName ::= SEQUENCE

{		
position		Position,
procedureName		ProcedureName
}		

PositionReport ::= SEQUENCE

{			
positioncurrent	[0]	Position,	
timeatpositioncurrent	[1]	Time,	
level	[2]	Level,	
fixnext	[3]	Position	OPTIONAL,
timeetaatfixnext	[4]	Time	OPTIONAL,
fixnextplusone	[5]	Position	OPTIONAL,
timeetaatdestination	[6]	Time	OPTIONAL,
remainingFuel	[7]	RemainingFuel	OPTIONAL,
temperature	[8]	Temperature	OPTIONAL,
winds	[9]	Winds	OPTIONAL,
turbulence	[10]	Turbulence	OPTIONAL,
icing	[11]	Icing	OPTIONAL,
speed	[12]	Speed	OPTIONAL,
speedground	[13]	SpeedGround	OPTIONAL,
verticalChange	[14]	VerticalChange	OPTIONAL,
trackAngle	[15]	Degrees	OPTIONAL,
heading	[16]	Degrees	OPTIONAL,
distance	[17]	Distance	OPTIONAL,
humidity	[18]	Humidity	OPTIONAL,
reportedWaypointPosition [19]	Position		OPTIONAL,
reportedWaypointTime	[20]	Time	OPTIONAL,
reportedWaypointLevel	[21]	Level	OPTIONAL
}			

PositionRouteClearanceIndex ::= SEQUENCE

{		
position	Position,	
routeClearanceIndex	RouteClearanceIndex	
}		

PositionSpeed ::= SEQUENCE

{		
position	Position,	
speed	Speed	
}		

PositionSpeedSpeed ::= SEQUENCE

{		
position	Position,	
speeds	SpeedSpeed	
}		

PositionTime ::= SEQUENCE

```

{
  position          Position,
  time              Time
}

```

PositionTimeLevel ::= SEQUENCE

```

{
  positionTime      PositionTime,
  level             Level
}

```

PositionTimeTime ::= SEQUENCE

```

{
  position          Position,
  times             TimeTime
}

```

PositionUnitNameFrequency ::= SEQUENCE

```

{
  position          Position,
  unitname          UnitName,
  frequency         Frequency
}

```

Procedure ::= IA5String (SIZE (1..20))**ProcedureName** ::= SEQUENCE

```

{
  type              [0]    ProcedureType,
  procedure         [1]    Procedure,
  transition        [2]    ProcedureTransition
}

```

OPTIONAL

ProcedureTransition ::= IA5String (SIZE (1..5))**ProcedureType** ::= ENUMERATED

```

{
  arrival           (0),
  approach          (1),
  departure         (2)
}

```

PublishedIdentifier ::= CHOICE

{			
fixName	[0]	FixName,	
navaid	[1]	Navaid	
}			

RemainingFuel ::= Time

RemainingFuelPersonsOnBoard ::= SEQUENCE

{		
remainingFuel	RemainingFuel,	
personsOnBoard	PersonsOnBoard	
}		

ReportingPoints ::= SEQUENCE

{			
latLonReportingPoints	[0]	LatLonReportingPoints,	
degreeIncrement	[1]	DegreeIncrement	OPTIONAL
}			

RevisionNumber ::= INTEGER (1..16)

RouteAndLevels ::= SEQUENCE

{		
routeOfFlight	RouteInformation,	
levelsOfFlight	LevelsOfFlight	
}		

RouteClearance ::= SEQUENCE

{			
airportDeparture	[0]	Airport	OPTIONAL,
airportDestination	[1]	Airport	OPTIONAL,
runwayDeparture	[2]	Runway	OPTIONAL,
procedureDeparture	[3]	ProcedureName	OPTIONAL,
runwayArrival	[4]	Runway	OPTIONAL,
procedureApproach	[5]	ProcedureName	OPTIONAL,
procedureArrival	[6]	ProcedureName	OPTIONAL,
routeInformations	[7]	SEQUENCE SIZE (1..128) OF RouteInformation	OPTIONAL,
routeInformationAdditional	[8]	RouteInformationAdditional	OPTIONAL
}			

RouteClearanceIndex ::= INTEGER (1..2)

-- RouteClearanceIndex identifies the position of the RouteClearance data
 -- in the ASN.1 type for
 -- ATC UplinkMessage, constrained Data, routeClearance Data
 -- ATC DownlinkMessage, constrained Data, routeClearance Data

RouteInformation ::= CHOICE

{		
publishedIdentifier	[0]	PublishedIdentifier,
latitudeLongitude	[1]	LatitudeLongitude,
placeBearingPlaceBearing	[2]	PlaceBearingPlaceBearing,
placeBearingDistance	[3]	PlaceBearingDistance,
aTSRouteDesignator	[4]	ATSRouteDesignator
}		

RouteInformationAdditional ::= SEQUENCE

{		
aTWAlongTrackWaypoints	[0]	SEQUENCE SIZE (1..8) OF ATWAlongTrackWaypoint OPTIONAL,
reportingpoints	[1]	ReportingPoints OPTIONAL,
interceptCourseFroms	[2]	SEQUENCE SIZE (1..4) OF InterceptCourseFrom OPTIONAL,
holdAtWaypoints	[3]	SEQUENCE SIZE (1..8) OF Holdatwaypoint OPTIONAL,
waypointSpeedLevels	[4]	SEQUENCE SIZE (1..32) OF WaypointSpeedLevel OPTIONAL,
rTARequiredTimeArrivals	[5]	SEQUENCE SIZE (1..32) OF RTARequiredTimeArrival OPTIONAL
}		

RTARequiredTimeArrival ::= SEQUENCE

{			
position	[0]	Position,	
rTATime	[1]	RTATime,	
rTATolerance	[2]	RTATolerance	OPTIONAL
}			

RTATime ::= SEQUENCE

{		
time		Time,
timeTolerance		TimeTolerance
}		

RTATolerance ::= INTEGER (1..150)

--unit= Minute, Range (0.1..15.0), resolution = 0.1

Runway ::= SEQUENCE

{		
direction		RunwayDirection,
configuration		RunwayConfiguration
}		

RunwayDirection ::= INTEGER (1..36)

RunwayConfiguration ::= ENUMERATED

```
{  
left           (0),  
right          (1),  
center         (2),  
none           (3)  
}
```

RunwayRVR ::= SEQUENCE

```
{  
runway          Runway,  
rVR             RVR  
}
```

RVR ::= CHOICE

```
{  
rVRFeet         [0]    RVRFeet,  
rVRMeters       [1]    RVRMeters  
}
```

RVRFeet ::= INTEGER (0..6100)

-- unit = Feet, Range (0..6100), resolution = 1

RVRMeters ::= INTEGER (0..1500)

-- unit = Meters (0..1500), resolution = 1

SecondsLatLon ::= INTEGER (0..59)

--unit = Second, Range (0.. 59), resolution = 1

Speed ::= CHOICE

```
{  
speedIndicated           [0]    SpeedIndicated,  
speedIndicatedMetric     [1]    SpeedIndicatedMetric,  
speedTrue                [2]    SpeedTrue,  
speedTrueMetric          [3]    SpeedTrueMetric,  
speedGround              [4]    SpeedGround,  
speedGroundMetric        [5]    SpeedGroundMetric,  
speedMach                [6]    SpeedMach  
}
```

SpeedIndicated ::= INTEGER (0..400)

-- unit = Knots, Range (0..400), resolution = 1

SpeedIndicatedMetric ::= INTEGER (0..800)

-- unit = Kilometers/Hour, Range (0..800), resolution = 1

SpeedGround ::= INTEGER (-50..2000)

-- unit = Knots, Range (-50..2000), resolution = 1

SpeedGroundMetric ::= INTEGER (-100..4000)

-- unit = Kilometers/Hour, Range (-100..4000), resolution = 1

SpeedMach ::= INTEGER (500..4000)

-- unit = Mach Range (0.5 to 4.0), resolution = 0.001

SpeedSpeed ::= SEQUENCE SIZE (2) OF Speed

SpeedTime ::= SEQUENCE

{	
speed	Speed,
time	Time
}	

SpeedTrue ::= INTEGER (0..2000)

-- unit = Knots, Range (0..2000), resolution = 1

SpeedTrueMetric ::= INTEGER (0..4000)

-- unit = Kilometers/Hour, Range (0..4000), resolution = 1

SpeedType ::= ENUMERATED

{	
noneSpecified	(0),
indicated	(1),
true	(2),
ground	(3),
mach	(4),
approach	(5),
cruise	(6),
minimum	(7),
maximum	(8),
...	
}	

SpeedTypeSpeedTypeSpeedType ::= SEQUENCE SIZE (3) OF SpeedType

SpeedTypeSpeedTypeSpeedTypeSpeed ::= SEQUENCE

{	
speedTypes	SpeedTypeSpeedTypeSpeedType,
speed	Speed
}	

Temperature ::= INTEGER (-100..100)

-- unit = Degree Celsius, Range (-100..100), resolution = 1

Time ::= SEQUENCE

```
{
  hours           TimeHours,
  minutes         TimeMinutes
}
```

TimeLevel ::= SEQUENCE

```
{
  time           Time,
  level          Level
}
```

TimeDeparture ::= SEQUENCE

```
{
  timeDepartureAllocated      [0]   Time OPTIONAL,
  timeDepartureControlled     [1]   ControlledTime OPTIONAL,
  timeDepartureClearanceExpected [2]   Time OPTIONAL,
  departureMinimumInterval    [3]   DepartureMinimumInterval OPTIONAL
}
```

TimeDistanceSpecifiedDirection ::= SEQUENCE

```
{
  time                               Time,
  distanceSpecifiedDirection         DistanceSpecifiedDirection
}
```

TimeDistanceToFromPosition ::= SEQUENCE

```
{
  time           Time,
  distance        Distance,
  tofrom         ToFrom,
  position        Position
}
```

Timehhmmss ::= SEQUENCE

```
{
  hoursminutes      Time,
  seconds           TimeSeconds
}
```

TimeHours ::= INTEGER (0..23)

-- unit = Hour, Range (0..23), resolution = 1

TimeUnitNameFrequency ::= SEQUENCE

```
{
  time           Time,
  unitName       UnitName,
  frequency      Frequency
}
```

TimeMinutes ::= INTEGER (0..59)

-- unit = Minute, Range (0..59), resolution = 1

TimePosition ::= SEQUENCE

```
{
  time           Time,
  position       Position
}
```

TimePositionLevel ::= SEQUENCE

```
{
  timeposition   TimePosition,
  level          Level
}
```

TimePositionLevelSpeed ::= SEQUENCE

```
{
  timeposition   TimePosition,
  levelspeed     LevelSpeed
}
```

TimeSeconds ::= INTEGER (0..59)

-- unit = Second, Range (0..59), resolution = 1

TimeSpeed ::= SEQUENCE

```
{
  time           Time,
  speed          Speed
}
```

TimeSpeedSpeed ::= SEQUENCE

```
{
  time           Time,
  speedspeed     SpeedSpeed
}
```

TimeTime ::= SEQUENCE SIZE (2) OF Time

TimeToFromPosition ::= SEQUENCE

```
{
  time           Time,
  tofrom         ToFrom,
  position       Position
}
```

TimeTolerance ::= ENUMERATED

```
{
  at              (0),
  atorafter      (1),
  atorbefore(2)
}
```

ToFrom ::= ENUMERATED

```
{
  to              (0),
  from           (1)
}
```

ToFromPosition ::= SEQUENCE

```
{
  toFrom         ToFrom,
  position       Position
}
```

TrafficType ::= ENUMERATED

```
{
  noneSpecified      (0),
  oppositeDirection (1),
  sameDirection      (2),
  converging         (3),
  crossing            (4),
  diverging           (5),
  ...
}
```

Turbulence ::= ENUMERATED

```
{
  light           (0),
  moderate        (1),
  severe          (2)
}
```

UnitName ::= SEQUENCE

```
{
  facilityDesignation [0] FacilityDesignation,
}
```

facilityName	[1]	FacilityName	OPTIONAL,
facilityFunction	[2]	FacilityFunction	
}			

UnitNameFrequency ::= SEQUENCE

{	
unitName	UnitName,
frequency	Frequency
}	

VersionNumber ::= INTEGER (0..15)

VerticalChange ::= SEQUENCE

{	
direction	VerticalDirection,
rate	VerticalRate
}	

VerticalDirection ::= ENUMERATED

{	
up	(0),
down	(1)
}	

VerticalRate ::= CHOICE

{	
verticalRateEnglish	[0] VerticalRateEnglish,
verticalRateMetric	[1] VerticalRateMetric
}	

VerticalRateEnglish ::= INTEGER (0..3000)

-- unit = Feet/Minute, Range (0..30000), resolution = 10

VerticalRateMetric ::= INTEGER (0..1000)

-- unit = Meters/Minute, Range (0..10000), resolution = 10

WaypointSpeedLevel ::= SEQUENCE

{		
position	[0] Position,	
speed	[1] Speed	OPTIONAL,
aTWLevels	[2] ATWLevelSequence	OPTIONAL
}		

WindDirection ::= INTEGER (1..360)

-- unit = Degree, Range (1..360), resolution = 1

Winds ::= SEQUENCE

```
{
  direction      WindDirection,
  speed          WindSpeed
}
```

WindSpeed ::= CHOICE

```
{
  windSpeedEnglish    [0]    WindSpeedEnglish,
  windSpeedMetric      [1]    WindSpeedMetric
}
```

WindSpeedEnglish ::= INTEGER (0..255)

-- unit = Knot, Range (0..255), resolution = 1

WindSpeedMetric ::= INTEGER (0..511)

-- unit = Kilometer/Hour, Range (0..511), resolution = 1

Year ::= INTEGER (1996..2095)

-- unit = Year, Range (1996..2095), resolution = 1

END